Säkerhetsrapport – SQLi Workshop

1. Inledning

Syftet med denna workshopen var att genomföra SQLi attacker mot ett REST API byggt i Node.js med SQLite. Därefter analysera sårbarheter och implementera åtgärder. I workshopen arbetade jag utifrån att bara API-endpoints och SQLite var känt. När workshopen var färdig utfördes en analys av backendkoden i index.js.

Under genomförandet hittades tre endpoints:

- **GET /api/products**
- **GET /api/products/:id**
- **DELETE /api/products/:id**

Alla dessa visade sig vara sårbara för SQLi (SQL-Injektion)

2. Attack 1 – Returnera alla tabeller och vyer

Attacken

Målet var att få databasen att visa vilka tabeller som fanns. Eftersom endpointen **/api/products/:id** byggde SQL frågan baserat på användar input utan någon Kontroll kunde jag injicera en UNION payload.

Payload

```
http://localhost:3003/api/products/1 UNION SELECT 1,name,sql,4 FROM
sqlite_master WHERE type='table'
```

Vad jag gjorde

1. Testade först felhantering genom att anropa /api/products/a, vilket returnerade “no such column: a”, vilket bekräftade att användarinput injicerades direkt i SQL.
2. Därefter testade jag en “boolean-bypass” **1 OR 1=1**, där alla produkter returnerades.
3. Sist använde jag UNION SELECT mot sqlite_master för att få fram tabellnamn och CREATE statements.

Resultat

Tabellerna som returnerades var följande:

Tabell	Kolumner
products	id, name, description, price
users	Id, firstName, lastName, email, password, userRole
sqlite_sequence	Systemtabell (name, seq)

Detta bekräftade att hela databasstrukturen var fullt åtkomlig.

Sårbarhet

Sårbarheten uppstår i följande del av koden:
`WHERE id = ${req.parms.id}`
Användarinput sätts in direkt i SQL strängen utan någon form av parameterhantering eller validering.

OWASP kategori

A05:2025 – Injection

Försvaret

Jag ersatte dynamisk strängbyggning med queries som använder parametrar:

```
app.get('/api/products/:id', (req, res) => {
      const stmt = db.prepare("SELECT * FROM products WHERE id = ?");
      const result = stmt.all(req.params.id);
      res.json(result);
});
```

Varför detta fungerar

Prepared statements skickar SQL koden och parametrarna separat. SQLite kan då inte tolka användarinput som SQL syntax.

Verifiering

Efter jag implementerat den nya koden så returnerar samma payload ett tomt resultat, inte tabellinformationen.

Riskbedömning

- **Likelihood**: Hög – en kombination av publik åtkomst och enkel exploatering
- **Impact**: Hög – hela databasstrukturen är fullt åtkomlig vilket möjliggör vidare attacker
- **Risknivå**: Kritisk

3. Attack 2 – Lista alla tabeller och datatyper

Attacken

Målet var att få fram alla kolumner och datatyper i databasen.

Payload

```
http://localhost:3003/api/products/1 UNION SELECT 1,'ALL COLUMNS
BELOW',group_concat(sql),4 FROM sqlite_master WHERE type='table'
```

Vad jag gjorde

Jag använde `group_concat(sql)` för att slå ihop alla CREATE statements till en sträng.

Resultat

Jag fick en komplett dump av databassens struktur, inklusive:

- Kolumnnamn
- Datatyper
- Primärnycklar
- constraints

Sårbarheten

Samma som i attack 1, användarinput sätts in direkt i SQL strängen utan någon form av parameterhantering eller validering.

OWASP kategori

A05:2025 – Injection

Försvaret

Samma fix som i attack 1, prepared statements.

Verifiering

Payloaden returnerar inte längre någon databasinformation.

Riskbedömning

- **Likelihood**: Hög
- **Impact**: Hög (databassdump är en förutsättning för större attacker)
- **Risknivå**: Hög

4. Attack 3 – Lista användare

Attacken

När tabellen users hade identifierats kunde jag extrahera användardata.

Payload

```
http://localhost:3003/api/products/1 UNION SELECT 1,firstName||'
'||lastName,email,4 FROM users
```

Och för lösenord (dock krypterade, antagligen med bcrypt)

```
http://localhost:3003/api/products/1 UNION SELECT 1,email,password,4
FROM users
```

Vad jag gjorde

Jag injicerade en UNION-select som matchade antalet kolumner i originalfrågan. Jag hittade antalet genom att göra en sekvens med `https://localhost:3003/api/products/1' ORDER BY 1-` där 1 stiger i antal för att identifiera kolumner i products tabellen. Använde UNION SELECT med X kolumner för att dumpa tabellnamn från sqlite_master.

Resultat

Jag fick ut:

- Förnamn
- Efternamn
- Email
- Hashade lösenord (bcrypt)

Även om lösenorden är hashade är läckan allvarlig

Sårbarheten

Samma SQL injektionsproblem som tidigare.

OWASP kategori

A05:2025 – Injection
A04:2025 – Cryptographic Failures (pga exponering av hashade lösenord)

Försvaret

Prepared statements + begränsning av vilka kolumner API får returnera.

Verifiering

Payloaden returnerar inte längre användardata (payloaden returnerar inget)

Riskbedömning

- **Likelihood**: Hög
- **Impact**: Mycket hög (personuppgifter + lösenord)
- **Risknivå**: Kritisk

5. Attack 4 – Radera alla produkter

Attacken

Målet var att radera alla produkter med en enda DELETE request.

Payload

```
await fetch('/api/products/5 OR 1=1', { method: 'DELETE' })
```

Skickades i consolen

Vad jag gjorde

Eftersom DELETE endpointen använde användarinput utan parameterbindning kunde jag injicera `OR 1=1`.

Reslutat

Alla produkter i databasen raderades.

Sårbarheten

I backend:

```
DELETE FROM products
      WHERE id = ${req.parms.id}
```

OWASP kategori

A05:2025 – Injection

Försvaret

Prepared statements:

```
app.delete('/api/products/:id', (req, res) => {
      const stmt = db.prepare("DELETE FROM products WHERE id = ?");
      const result = stmt.run(req.params.id);
      res.json(result);
});
```

Man borde även lägga till någon form av AUTH + RBAC för att göra så att enbart en Admin kan använda sig utav DELETE. För i nuläget kan man fortfarande använda sig av await fetch('/api/products/1',{method:'DELETE'}).then(r=>r.text()).then(console.log) och ta bort en produkt åt gången, eller skriva en script som ändrar '1' till X+1 varje gång scriptet körs. Då raderas alla produkter ändå, för att lösa detta kan man implementera rate limiting som hade kunnat se ut såhär:

```
const rateLimit = require('express-rate-limit')

const deleterLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,  // 15 min
  max: 3,                     // Max 3 DELETE per IP
  message: 'Too many delete attempts'
})

app.delete('/api/products/:id', deleterLimiter, (req, res) => {
})
```

Att köra detta tillsammans med monitoring kan säkerställa att angriparen bara har tid att radera max 3 produkter innan man märker och kan åtgärda det.

Verifiering

Payloaden raderar nu endast en produkt, aldrig alla

Riskbedömning

- **Likelihood**: Hög
- **Impact**: Hög (dataförlust)
- **Risknivå**: Hög

6. Flerlagrat försvar

Utöver prepared statements rekommenderar jag:

Inputvalidering

- Endast numeriska IDn accepteras

Least Privilege

- Separat databas för användare med endast SELECT/DELETE på products
- Ingen åtkomst till users från produkt API

Rate limiting

- Begränsa antal requests per IP

API design

- Exponera aldrig endpoints som returnerar hela SQL fel
- Undvik att returnera råa databasobjekt

Monitoring

- Logga misstänkta mönster som OR 1=1, UNION, –.

7. Sammanfattning

Rangordning av sårbarheter (allvarlighetsgrad)

1. Dump av användare (kritisk)
2. Mass delete (hög)
3. Databas dump (hög)
4. Tabell enumerering (medel)

Rekommendationer

- Inför prepared statments – eliminerar samtliga SQLi attacker
- Implementera inputvalidering för att sätta stop för felaktiga ID format
- Inför rollbaserad åtkomst så att produkt API inte kan läsa användartabellen
- Aktivera loggning och övervakning för att upptäcka och stoppa attacker
- Utbilda inom säker kod och OWASP Top 10

Slutsats

Applikationen var till en början helt oskyddad mot SQLi, vilket gjorde det möjligt att totalt kompromisera databasen. Genom att införa prepared statements, inputvalidering och flerlagrat försvar försvinner sårbarheterna och API blev säkrare.

Säkerhetsrapport – XSS Workshop

1. Inledning

Syftet med denna workshop var att granska säkerheten av ett system under utveckling. Systemet inehöll registrering, inloggning och två användarroller: user och superadmin.

Attacken gjordes stegvis

- Analys av REST trafik genom webbutvecklarverktyg och Thunderclient
- Analys efter första attacken genomförts, verifiera sårbarhet och förslag på förbättring.

Mål

- Genomföra en XSS attack som påverkar superadmin
- Eskalera privilegier på en user till superadmin via REST API

2. Attack 1 – XSS attack mot superadmin

Målet med den första attacken var att injicera skadlig JS som körs när supoeradmin loggar in. När superadmin loggar in får de automatiskt en lista över alla användare, detta kunde då exploiteras genom att skapa en ny användare med payload som användarens namn.

Payload

Vid registrering av ny användare angavs följande som “Name”:

```
<img src=x onerror="location.href='https://www.expressen.se'">
```

Vad jag gjorde

1. Registrerade en ny användare med ovanstående payload som namn.
2. Loggade ut och testade logga in som superadmin
3. Superadminpanelen laddade en lista över användare och tolkade payloaden som HTML innehåll
4. `onerror` triggas eftersom `src=x` inte är en giltig bild.
5. JS körs och superadmin blir omdirigerad till Expressen.se

Resultat

Attacken fungerade. Superadmin kunde inte längre använda systemet eftersom varje inloggning blev omdirigerad.

Detta bevisar att:

- Användardata renderas osanerat i HTML
- Systemet saknar output encodring
- Systemet saknar CSP (Content Security Policy)

Sårbarheten

Sårbarheten är en **Stored XSS**

- Skadlig kod lagras i databasen
- Renderas för andra användare (alla superadmins)
- Körs i deras webbläsare

Sårbara delar i koden:

```
runQuery(name, req, res, req.body, `
  INSERT INTO $$   {name} (   $${Object.keys(req.body)})
  VALUES (${Object.keys(req.body).map(x => ':' + x)})
`);
```

```
html += '<td class="' + typeof value + '">' + value + '</td>';
```

OWASP kategori

A05:2025 – Injection
A02:2025 – Security Misconfiguration (saknar CSP)

Försvaret

Följande åtgärder implementeras:

1. Output encoding

Vid rendering av användardata ska HTML specialtecken ersättas:

```
function escapeHTML(str) {
      return str
      .replace(/&/g, "&amp;")
      .replace(/</g, "&lt;")
      .replace(/>/g, "&gt;")
      .replace(/"/g, "&quot;")
      .replace(/'/g, "&#039;");
```

```
}
```

2. Input validering vid registrering

```
if (name === userTable) {
    const nameRegex = /^[a-zA-ZåäöÅÄÖ\s\-']+$/;
    if (!nameRegex.test(req.body.firstName) ||
!nameRegex.test(req.body.lastName)) {
      res.status(400);
      res.json({ _error: 'Invalid name: Only letters, spaces, hyphens,
and apostrophes allowed.' });
      return;
    }
    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    if (!emailRegex.test(req.body.email)) {
      res.status(400);
      res.json({ _error: 'Invalid email format.' });
      return;
    }
```

Ändringen sker i rest-api.js, i POST handlern för /api/costumers.
Detta fungerar utmärkt eftersom det förhindrar injektion av skadlig kod. Enligt OWASP XSS Prevention Cheat Sheet är strikt allow list validering mer effektivt än en blacklist, eftersom angripare kan obfuscera kod. Så genom att begränsa till alfanumeriska tecken och vanliga namn tecken blockeras all HTML/JS. Medans man fortfarande tillåter namn som Anna-Karin och O'Learys.

3. Content Security Policy

Eftersom Workshopen faller under **A02:2025 – Security Misconfiguration** (saknar CSP) kan det vara ett alternativ att lägga till en policy. Jag testade med denna:

```
Content-Security-Policy: default-src 'self'; script-src 'self'
'unsafe-inline'; style-src 'self' 'unsafe-inline'; img-src 'self' data:;
connect-src 'self'; form-action 'self'; frame-ancestors 'none';
```

Valet är baserat på:

- Balans – Policyn begränsar 'self' mentillåter 'unsafe-inline' eftersom Workshopen använder inline-script och inline-CSS.
- Skydd mot XSS – Även om policyn inehåller 'unsafe-inline' stoppar den farliga attacker som laddar externa scripts, exxfiltrerar data eller kör kod från andra domäner, bryter persistent XSS.
- OWASP risker – Direktiven default-src 'self', connect-src 'self', frame-ancestors 'none' och form-action 'self' minskar risken för injektionj, dataexfiltration mfl. Det är ett enkelt men effektivt lager i säkerhets strategin.

Verifiering

Efter fixarna:

- Superadmin omdirigeras inte
- Kan inte skapa användare med samma payload
- Om payload lyckas komma förbi renderas den som text inte HTML

Riskbedömning

- **Likelihood**: Hög
- **Impact**: Hög (superadmin låses ute)
- **Risknivå**: Kritisk

3. Attack 2 – Privilege Escalation

Attacken

Målet var att undersöka ifall en vanliga användare kunde ändra sin egen roll via API

Systemet hade en endpoint som vi fick reda på i Workshopen:

```
PUT /api/edit-my-user-info
```

Vad jag gjorde

1. Registrerade en ny användare
2. Loggade in och öppnade Network → Perserve log.
3. Inspekterade login responsen:

```
{
  "id": 20,
  "email": "superman@hero.com",
  "firstName": "Super",
  "lastName": "Man",
  "userRole": "user"
}
```

4. Testade sedan skick en PUT request direkt till API genom ThunderClient

Payload

```
PUT http://localhost:3010/api/edit-my-user-info
```

Body:

```
{
  "userRole": "superadmin"
}
```

Resultat

API accepterade ändringen. Min nyskapade användare med standard roll user blev, Superadmin (BAC).

Sårbarheten

Backend kontrollerar inte:

- Om användaren får lov att ändra sin egen roll
- Om 'userRole' borde kunna ändras, och om det borde kunna ändras a user
- Varifrån requesten kom

Sårbarheten i koden ligger vid: (Ingen check på userRole innan update)

```
runQuery('edit-my-user-info', req, res, queryParameters, `
  UPDATE customers
  SET ${Object.keys(req.body).map(x => x + ' = :' + x)}
  WHERE id = :id
`);
```

OWASP kategori

A01:2025 - Broken Access Control

Försvaret

Följande åtgärd implementerades:

1. Skydd av känsliga fält

För att förhindra privilege escalation blockeras uppdateringar av fälten userRole och id i PUT förfrågningar. Endast tillåtna fält får ändras, firstName, lastName osv. Försöker man ändra userRole eller id får returneras 403 Forbidden.

```
const forbiddenFields = ['userRole', 'id'];
for (let field of forbiddenFields) {
  if (req.body.hasOwnProperty(field)) {
    res.status(403);
    res.json({ _error: `Forbidden: Cannot update protected field
'${field}'.` });
    return;
  }
}
```

Man kan här bygga vidare med RBAC och göra så att endast en spcifik roll har tillgång att ändra userRole och id, eller andra om man lägger till.

Detta kan man uppnå med RBAC genom IF statments och sedan bygga vidare.

```
if (req.user.role !== "superadmin") {
  return res.status(403).json({ error: "Forbidden" });
}
```

Verifiering

Efter fixen

- Användare kan inte ändra userRole eller ID
- Returnerar 403 Forbidden

Riskbedömning

- **Likelihood**: Hög
- **Impact**: Extrem (full systemkompromettering)
- **Risknivå**: Kritisk

4. Flerlagrat försvar

Inputvalidering

- HTML taggar förbjuds i alla textfällt
- Strikt email validering

Output encoding

- All användardataHTML escapas innan rendering

Content Security Policy

- Förhindrar inline script och onerror attack

Åtkomstkontroll

- Backend måste alltid verifiera användarroll
- Endast superadmins får ändra specifika saker, exempel roller.

Least Privilege

- Vanliga användare får endast ändra namn, mail, lösenord
- userRole får aldrig komma från klienten

Loggning och Monitoring

- Logga ändringar av roller och användaruppgifter
- Alert vid misstänksamma requests

5. Sammanfattning

Rangordning av sårbarheter (allvarlighetsgrad)

1. Privilege Escalation (BAC) - Kritisk
2. Stored XSS - Kritisk
3. Security Misconfiguration (saknade CSP) - Hög
4. Bristane inputvalidering - Medel

Rekommendationer

- Implementera rollbaserad åtkomstkontroll (RBAC)
- Inför output encoding och CSP för att stoppa XSS
- Begränsa vilka fält somfår ändras via API

- Inför loggning och övervakning ava alla relevanta ändringar
- Utbilda inom säker kod och OWASP Top 10

Slutsats

Systemet var mycket sårbart, vanliga användare kunde:

- Omdirigerasuperadmins med XSS
- Förhöja sin egen roll till superadmin

Efter implementerade säkerhetsåtgärder blev systemet mer robust och säkert.

Säkerhetsrapport – Traversal Workshop

1. Inledning

I denna Workshop skulle en hemsida undersökas för sårbarheter. Hemsidan var utvecklad av en oerfarenutvecklare och skapad utan säkerhetsåtgärder vilket möjliggjorde för traversal attacker.

Målet var att se ifall en angripare kunde:

- Lista innehållet i samtliga mappar på servern
- Läsa filer utanför vad som syns på frontend
- Göra en riskbedömning och förslå säkerhetsåtgärder

2. Attack 1 – Directory listing via Traversal

Attacken

När jag browsade och undersökte hemsidan hittade jag i dogs.html en Request Initiator Chain som avslöjade en dold endpoint: `/api/photolist?folder=dogs`
Denna endpoint returnerade en lista över filer i mappen dogs/

För att testa traversal använde jag följande:

```
http://localhost:3002/api/photolist?folder=../
```

Vad jag gjorde

1. Öppnade DevTools → Network
2. Navigerade till dogs.html
3. Såg att sidan laddade bilder via `/api/photolist?folder=dogs`
4. Ändrade folder parametern till `../`
5. Fick nu listor över filer i mappen ovanför `photos/`
6. Fortsatte gå back ännu mer med `/api/photolist?folder=../../` och kunde då lämna hela workshop mappen

Resultat

Jag kunde lista innehållet i:

- Projekt mappen
- Andra mappar på min dator

Detta bevisade en traversal sårbarhet

Sårbarheten

Backend validerade inte folders parametern och använde den direkt i filsökvägen, osaneradinput i sökvägen:

```
const list = fs.readdirSync(path.join(import.meta.dirname, 'images',
req.query.folder));
```

OWASP kategori

A01:2025 - Broken Access Control
A02:2025 - Security Misconfiguration

Försvaret

Jag implementerade följande:

1. Path normalisering

```
const BASE_DIR = path.join(import.meta.dirname, 'images');  // Definiera
basmapp
// I endpoint:
const folder = req.query.folder || '';
const fullPath = path.resolve(BASE_DIR, folder);
if (!fullPath.startsWith(BASE_DIR + path.sep)) {
  return res.status(403).json({ error: 'Forbidden: Invalid path' });
}
```

Genom att normalisera sökvägen med path.resolve tas alla relativa delar så som ../ bort. Detta gör att vi alltid får en absolut sökväg. Därefter kontrolleras det att den resulterande sökvägen fortfarande ligger innanför den avsedda images mappen, jämför prefixet med BASE_DIR.

Detta gör att användaren inte kan navigera utanför tillåten mapp.

2. Explicit input validering

```
// Lägg till efter folder definition i endpoint:
if (!folder || /[^\w/-]/.test(folder)) {
  return res.status(400).json({ error: 'Bad Request: Invalid folder
parameter' });
}
```

Validering av parametrarna direkt när den tas emot gör att ogiltig eller misstänkt input stoppas innan servern gör några filåtkomster eller annan onödig processing. Detta följer principerna i OWASPs Input Validation riktlinjer och minskar risken för felaktiga anrop och resursförbrukande attacker.
Regeln begränsar vilka tecken som tillåts i folder parametern och kan därmed förhindra exempelvis tomma värden eller oväntade symboler. Denna kontroll funkar som ett extra skydd ovanpå Path normaliseringen.

3. Allowlist för undermappar

```
const ALLOWED_FOLDERS = ['dogs', 'cats'];  // Definiera globalt eller i
endpoint

// Lägg till efter validering i endpoint:
const normalizedFolder = path.basename(fullPath).replace(BASE_DIR, '');
if (!ALLOWED_FOLDERS.includes(normalizedFolder) && normalizedFolder !==
'') {
  return res.status(403).json({ error: 'Forbidden: Folder not allowed'
});
}
```

Med hjälp av allow list begränsas åtkomsten till ett antal godkända undermappar. Detta är en mycket säkrare strategi än att försöka blockera (blacklist) otillåtna namn.

Verifiering

- ../ och ../../ blockeras via normalisering
- URL kodade varianter som %2e%2e/ blockeras
- Ogiltiga eller tomma parametrar returnerar 400
- Endast mappar i ALLOWED_FOLDERS kan listas, annat får 403

Riskbedömning

Likelihood: Hög (vanlig attackvektor i filhanterande API)
Impact: Mycket hög (potentiell extrahering av källkod, konfigurationsfiler, användar data med flera – brytande konfidentialitet)
Risknivå: Kritisk (kan leda till full systemkompromiss)

2. Attack 2 – Fil läsning via /api/photo?file=

Attacken

När jag fortsatte analysera hemsidan hittade jag en bild på en hund som i Network filken visade att bilden laddades genom:

```
/api/photo?file=dogs/dog1.webp
```

Eftersom detta var en file= parameter testade jag att använda traversal här.

Payload

För att kunna läsa projektets backend kod testade jag (efter att jag med tidigare map traversal hittat index.js) följande:

```
http://localhost:3002/api/photo?file=../index.js
```

Vad jag gjorde

1. Identiferade endpointen `/api/photo?file=`
2. Testade traversal
3. Servern returnerade innehållet i index.js
4. Testade fler filer så som `README.md` och `package.json`

Resultat

Jag kunde läsa:

- Projekt backend kod
- Konfigurationsfiler

Detta visar på en fullständig file read sårbarhet

Sårbarheten

Sårbarheten uppstår i backend som inte validerar file parametern och använder den direkt i filsökvägen:

```
res.sendFile(path.join(import.meta.dirname, 'images', req.query.file));
```

OWASP kategori

A01:2025 - Broken Access Control
A02:2025 - Security Misconfiguration

Försvaret

Jag implementerade följande åtgärder:

1. Path normalisering

```
const fullPath = path.resolve(BASE_DIR, file);
if (!fullPath.startsWith(BASE_DIR + path.sep)) {
  return res.status(403).json({ error: 'Forbidden: Invalid path' });
}
```

Normaliserar sökvägen för att ta bort relativa sekvensker och verifierar confinement

2. Explicit input validering

```
if (!file || /[^\w\.-]/.test(file) || file.includes('..')) {  // Tillåt
alfanumeriska, ., -; blockera '..'
  return res.status(400).json({ error: 'Bad Request: Invalid file
parameter' });
}
```

Kontrollerar parameterns gilgithet för att blockera ogiltig eller skadlig input, minskar processing risker enligt OWASP Input Validation Cheat Sheet.

3. Allowlist för filtyper eller namn

```
const ALLOWED_EXTENSIONS = ['.jpg', '.png', '.webp'];
const fileExt = path.extname(file).toLowerCase();
if (!ALLOWED_EXTENSIONS.includes(fileExt)) {
  return res.status(403).json({ error: 'Forbidden: File type not
allowed' });
}
```

Begränsar till vissa tillåtna fiuler eller typer, t.e.x endast bilder, säkrare än att försöka blacklista enligt OWASP.

Verifiering

Efter implementerade fixar:

- ../index.js och %2e%2e blockeras med 403
- Ogiltiga filer (tomt eller förbjuda tecken) returnerar 400
- Endast tillåtna filtyper hämtas, andra nekas med 403.

Riskbedömning

Likelihood: Hög
Impact: Mycket hög (dataextrahering)
Risknivå: Kritisk

3. Flerlagrat försvar

Kontroll av root katalog

`startsWith(BASE_DIR)` säkerställer att sökvägen inte lämnar bildmappen.

Minimala rättigheter

Servern ska inte ha åtkomst till känsliga filer.

Utökad allowlist

Enbart godkännda mappar och filtyper ska tillåtas

Loggning

Alla traversal försök bör loggas

4. Sammanfattning

Rangordning av sårbarheter (allvarlighetsgrad)

1. Fil läsning via /api/photo?file= (kritisk)
2. Directory traversal via /api/photolist?folder= (kritisk)
3. Saknad path normalisering (hög)
4. Saknad inputvalidering (medel)

Rekommendationer

- Implementera path normalisering och root kontroll
- Inför allowlist för mappar och filtyper
- Begränsa serverns filrättigheter
- Lägg till loggning och övervakning av traversal försök
- Utbilda inom säker kod och OWASP Top 10

Slutsats

Hemsidan var från början helt oskyddad mot traversal attacker vilket möjligjorde åtkomst till både projektfiler och systemfiler. Efter införda säkerhetsåtgärder är filhanteringen betydligt säkrare och följer standard säkerhetsprinciper.

Säkerhetsrapport - Monkeypatching MITM

1. Inledning

I denna workshop skulle jag undersöka hur sårbar en webbshop var när affärslogiken ligger i frontend och klienten ansvarar för att:

- Kontrollera rabattkoder
- Beräkna rabatter
- Hämta produktpriser
- Presentera slutpriser

Målet var att:

1. Visa 20% rabatt utan att känna till rabattkoden
2. Sätta vilket pris jag vill på alla produkter
3. Diskutera om detta är en säkerhetsrisk och om man kan förhindra

Attacken bygger på Monkeypatching av fetch(), man ersätter webbläsarens inbyggda fetch med en egen version som manipulerar svaren som kommer från servern.

Källa till attackmetoderna som användes:
[Workshop 4: Förslag på monkey-patchning av fetch som fungerar]

2. Attack 1 – Visa 20% rabatt

Attacken

Hemsidan kontrollerade rabattkoder genom API anrop:

```
/api/check-rebate-code/<kod>
```

Frontend förlitar sig helt på svaret från detta API
Om API svarar med `{ ok: true }` visas rabatten.

Då kontrollen sker i frontend kan vi manipulera svaret innan det når applikationen

Payload

```
 let oldFetch = fetch;
fetch = async function (...args) {
  let [url, options] = args;
  let rawResult = await oldFetch(...args);

  let oldJson = rawResult.json;
  rawResult.json = async function () {
    let data = await oldJson.apply(rawResult);

    if (url.startsWith('/api/check-rebate-code/')) {
      data = { ok: true }; // Tvinga rabattkod att vara giltig
    }

    return data;
  };

  return rawResult;
};
```

Vad jag gjorde

1. Öppnade webbsidan
2. Öppnade DevTools → Console
3. Klistrade en monkeypatch koden
4. Testade valfri rabattkod t.e.x huraRabatt
5. Frontend trodde nu att rabattkoden var giltig
6. Priserna visades med rabatt på 20%

Resultat

Jag kunde aktivera rabatten utan att känna till den riktiga rabattkoden.

Förklaring av vad koden gör

- oldFetch sparar orginal fetch
- Jag ersätter fetch med egen funktion
- När sidan anropar /api/check-rebate-code/…
  → manipuleras JSON svaret
  → returnerar { ok: true } oavsett vad servern svarar
- Frontend tror att rabattkoden är krorrekt
- Rabatten visas

Sårbarheten

All rabattlogik ligger i frontend.
Severn verfiferar inte rabatten

OWASP kategori

A01:2025 - Broken Access Control
A06:2025 - Insecure Design

Riskbedömning

Likelihood: Hög
Impact:Hög
Risknivå: Hög

3. Attack 2 – Sätt valfritt pris på alla produkter

Attacken

Produktpriserna hämtas från:
`/product-data.json`
Frontend använder dessa priser direkt för att visa totalsummor.
Genom att monkeypatcha fetch() kunde jag manipulera produktlistan innan den nådde applikationen.

Payload

```
      let oldFetch = fetch;
fetch = async function (...args) {
  let [url, options] = args;
  let rawResult = await oldFetch(...args);

  let oldJson = rawResult.json;
  rawResult.json = async function () {
    let data = await oldJson.apply(rawResult);

    if (url === '/product-data.json') {
      for (let product of data) {
        product.price *= 0.1; // Sänk priset till 10%
      }
    }
```

```
    return data;
  };

  return rawResult;
};
```

Vad jag gjorde

1. Öppnade DevTools → Console
2. Klistrade in monkeyoatch koden
3. Skriv in valfri rabattkode och klicka ok eller klicka på cancel när den frågar om rabattkod
4. Alla priser visades nu med 90% rabatt

Resultat

Jag kunde nu sätta vilket pris jag ville på alla produkter genom att ändra `product.price *= 0.1;` till vad jag ville (0.5 (50% rabatt))

Förklaring av vad koden gör

- När sidan hämtar /product-data.json fångar jag upp svaret
- Loppar igenom alla produkter och ändrar product-price innan frontend får datan
- Frontend visar de manipulerade priserna

Sårbarheten

Frontend litar på datan den får av klienten
Backend för ingen pris verfiering.

OWASP kategori

A01:2025 - Broken Access Control
A06:2025 - Insecure Design

Riskbedömning

Likelihood: Hög
Impact:Hög till Kritisk
Risknivå: Hög

4. Diskussion – Är detta en säkerhetsrisk?

Farligt att kunna manipulera priser i frontend?

Både Ja och Nej

- Ja om backend litar på vad frontend säger
  → Angriparen kan som vi gjorde manipulera priserna och lägga en order till rabbaterat pris.
- Nej om backend gör en slut validering (kontroll)
  → då spelar det ingen roll vad som visas i webbläsaren

Kan man undvika att frontend manipuleras?

Nej.
Det går inte att förhindra att en användare manipulerar JS i sin egen webbläsare.

CSP hjälper inte.
Att frysa fetch hjälper inte.
Verktyg som TamperMonkey kan alltid köra kod innan sidan laddas.

Kan man förhindra att order med fel pris läggs?

Ja, backen måste göra en slutkontroll.

Backend borde:

- Hämta produktpriser från databasen
- Strunta i priser som skickas av klienten
- Räkna ut totalsumman i backend
- Valideras rabattkoder serverside

Frontend kan manipuleras och ska aldrig avgöra vad som gäller.

5. Sammanfattning

Rangordning av sårbarheter (allvarlighetsgrad)

1. Manipulation av produktpriser (kritisk om backend validering saknas)
2. Manipulation av rabattkod (krritisk om backend validering saknas)
3. Frontend beroende logik (hög)

Rekommendationer

- Flytta affärslogik till backend
- Backend ska alltid räkna ut slutpriser
- Backend ska validera rabattkoder
- Backend ska ignorera priser som skickas från klienten
- Logga försök och misstänkta avvikelser

Slutsats

Monkeypatching av fetch() visar att frontend aldrig ska ses som säker. Det är omöjligt att skydda klienten från manipulation och måste därför:

- Backend ska vara den “sanna källan” för all affärslogik och data
- Alla prsier, rabatter och totalsummor ska beräknas serverside

Det är inte möjligt att säkra upp mot monkeypatching i sig men det gör att bygga upp ett system där monkeypatching inte har någon betydelse.